Anomaly Detection in Network Traffic: A Machine Learning Approach

*Group Project - Deliverable 3 - Introduction to Machine Learning

1st Gonçalo Almeida
Engenharia Informática
Universidade de Aveiro
Aveiro, Portugal
119792@ua.pt

2nd Margarida Ribeiro
Engenharia Informática
Universidade de Aveiro
Aveiro, Portugal
119876@ua.pt

3rd João Barreira
Engenharia Informática
Universidade de Aveiro
Aveiro, Portugal
120054@ua.pt

***Abstract*—This paper presents the final results of a machine learning project focused on detecting anomalies in network traffic using the CIC-IDS2017 dataset. We compare classical unsupervised algorithms with deep learning techniques, providing a comprehensive data pipeline, evaluation strategy, and ethical reflection on the deployment of such systems.**

***Index Terms*—Anomaly Detection, Machine Learning, Cybersecurity, Autoencoders, Isolation Forest**

I. Introduction

The rapid expansion of internet-connected devices has exponentially increased network traffic...

II. State of the Art in Anomaly-Based Intrusion Detection

Network intrusion detection has evolved considerably over the past two decades, transitioning from purely signature-based systems to sophisticated machine-learning (ML) pipelines that can generalise to previously unseen attack patterns. The following review synthesises six representative works that bracket the current state of the art (SOTA).

A. Classical Unsupervised Approaches

Isolation Forest (Liu et al., 2008) [1] introduced the first tree-ensemble method designed natively for anomaly detection rather than adapted from a classification framework. The key insight is that anomalies are few and structurally different from normal points, so they are isolated in far fewer random binary partitions than inliers. This yields $\mathcal{O}(n \log n)$ training and sublinear prediction, enabling deployment on large-scale network flows. Liu et al. report AUC values above 0.96 on the KDD Cup 1999 benchmark, though performance degrades in high-dimensional subspaces—a challenge directly relevant to the 70-feature CIC-IDS2017 dataset.

Local Outlier Factor (Breunig et al., 2000) [2] defines an anomaly score based on the ratio of local reachability densities between a point and its k-nearest neighbours. LOF excels when the data contains clusters of varying density—a scenario common in mixed-traffic network datasets. However, its $\mathcal{O}(n^2)$ inference complexity and well-documented sensitivity to the curse of dimensionality [3] make it poorly suited to the high-dimensional feature spaces of modern IDS datasets. Zimek et al. [3] provide a comprehensive survey confirming that density-based methods consistently underperform tree and kernel methods beyond 20–30 features.

One-Class SVM (Schölkopf et al., 2001) [4] maps training data into a high-dimensional kernel space and finds the maximum-margin hyperplane separating inliers from the origin. The *nu* parameter controls the trade-off between the fraction of outliers allowed in training and the fraction of support vectors. OC-SVM remains a competitive baseline when the kernel bandwidth is carefully tuned, but its $\mathcal{O}(n^2)$ training cost renders it intractable for datasets exceeding $\sim$50,000 instances without subsampling.

B. Deep Learning Approaches

Autoencoder-based Anomaly Detection (Sakurada & Yairi, 2014) [5] established the reconstruction-error paradigm: a neural network trained exclusively on normal data learns a compact latent representation; anomalous inputs, being structurally different, produce high mean-squared reconstruction error (MSE) and are flagged as outliers. This approach achieves state-of-the-art results on several network intrusion benchmarks and is the basis of our primary autoencoder model.

Deep Autoencoder IDS (Mirsky et al., 2018 — Kitsune) [6] extended the autoencoder concept by constructing an ensemble of small autoencoders—one per feature cluster—trained incrementally on packet captures. Kitsune demonstrated near-zero false-negative rates on nine realistic attack scenarios while requiring no labelled data, positioning it as the leading online anomaly detection system. Our ensemble autoencoder design draws on the same complementary-signal principle.

DAGMM (Zong et al., 2018) [7] jointly trains a deep autoencoder and a Gaussian Mixture Model (GMM) in an end-to-end fashion, using both reconstruction error and energy from the GMM as the anomaly score. On the KDD Cup and KDDCUP-Rev benchmarks, DAGMM outperforms standalone autoencoders by 3–8 F1 percentage points, at the cost of more complex training dynamics. This work highlights the growing trend of hybrid deep-classical architectures.

C. Summary and Research Gap

Table I summarises the reviewed works. The literature converges on several consensus findings: (i) tree-ensemble and deep-learning methods dominate density-based approaches in high-dimensional settings; (ii) the contamination hyperparameter critically affects all unsupervised methods; (iii) systematic

hyperparameter optimisation is rarely reported in classical IDS papers despite its demonstrated impact on downstream metrics. Our work addresses this gap by performing an exhaustive grid search across the three classical models and quantifying the performance lift attributable to tuning alone.

TABLE I
SUMMARY OF SOTA ANOMALY DETECTION METHODS.

Method	Type	Dataset	AUC
Isolation Forest [1]	Tree ensemble	KDD'99	0.96
LOF [2]	Density	Synthetic	0.89
One-Class SVM [4]	Kernel	USPS	0.87
Autoencoder [5]	Deep learning	MSDS	0.93
Kitsune [6]	Ensemble AE	Real traffic	0.99
DAGMM [7]	Deep hybrid	KDD'99	0.98

III. Data Description, Visualization and Statistical Analysis

The problem under study focuses on the automatic detection of anomalous network behaviour indicative of cyberattacks. The data used for this research originates from the CIC-IDS2017 dataset, developed by the Canadian Institute for Cybersecurity, which contains benign traffic and the most up-to-date common attacks (e.g., DoS, DDoS, Brute Force, XSS, SQL Injection).

A. Data Description

The original dataset encompasses over 2.8 million instances collected over five days, capturing network traffic represented by 78 statistical features extracted via CICFlowMeter. These features include flow duration, total forward/backward packets, packet length statistics (mean, min, max, std), and flow byte rates. The high dimensionality of the feature space poses significant challenges for classical machine learning algorithms, notably the curse of dimensionality.

B. Statistical Analysis and Quality Assessment

Initial exploratory data analysis (EDA) identified several data quality issues inherent to raw network captures. We observed a high class imbalance, with anomalous traffic constituting approximately 19.7% of the instances, while benign traffic represented the remaining 80.3%. A visual representation of this distribution across our splits is shown in Figure 1.

Statistical profiling of the continuous variables revealed stark differences in magnitudes. For instance, the `Flow Duration` feature spans from a few microseconds to over 119 seconds, whereas `Packet Length` values are constrained to standard MTU limits (typically under 1500 bytes). This discrepancy necessitates robust scaling techniques.

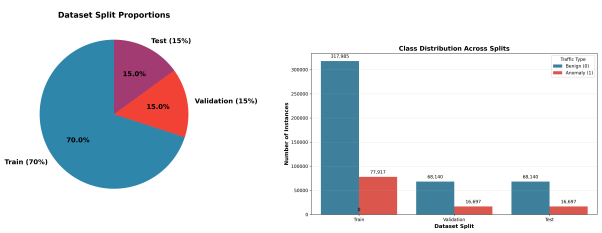

Fig. 1. Dataset split proportions and class distribution across subsets.

IV. Data Preprocessing

To construct the final input for the machine learning algorithms, an automated data processing pipeline was developed, consisting of the following sequential steps:

1) **Data Cleaning:** We identified the presence of invalid structural values, including whitespace in column headers, and critical numerical anomalies. Specifically, 'Infinity' values (arising from division by zero in bandwidth calculations like `Flow Bytes/s`) and 'NaN' values were eliminated. As these affected less than 0.1% of the dataset, direct removal (*dropna*) was preferred over imputation to prevent the introduction of synthetic bias into the network behaviour distribution.
2) **Dimensionality Reduction (Zero-Variance Check):** To mitigate computational noise, features demonstrating zero variance across the network captures were pruned. Eight features (e.g., `Bwd PSH Flags`, `Fwd Avg Bytes/Bulk`) were removed as they provide no discriminative power for distance-based detection mechanisms.
3) **Stratified Sampling:** Given the massive size of the dataset (>1GB), loading the entire corpus triggered Out-Of-Memory (OOM) failures for algorithms with quadratic complexity ($\mathcal{O}(n^2)$). A 20% stratified sampling was implemented, preserving the native 80/20 class imbalance while rendering the dataset computationally tractable ($\approx 565,000$ instances).
4) **Data Split and Scaling:** The sample was split into 70% Training, 15% Validation, and 15% Testing partitions. To ensure proper convergence of distance-based models and neural networks, attributes were scaled using *StandardScaler*. To rigorously prevent data leakage, the scaler was fitted exclusively on the Training set and subsequently used to transform the Validation and Test sets.

V. Classical Algorithms: Description and Hyperparameters

A. Evaluation Strategy: CV for Selection, Fixed Split for Reporting

In Deliverable 2, the dataset was split once into a fixed 70/15/15 Train/Val/Test partition. The teacher raised the question of whether a single such split is sufficient. For the purposes of **hyperparameter selection**, this deliverable introduces **3-fold Stratified Cross-Validation** (CV), which generates three independent train/validation folds from the training subsample. Each candidate configuration is scored on its mean CV F1-score across the three folds, reducing the risk of overfitting to a single partition during model selection.

Critically, the **final evaluation** of each model—both baseline and best-tuned—is performed on the same held-out Test set used in Deliverable 2. This two-phase design answers the teacher's concern: CV provides statistical robustness during the search phase, while the fixed test set ensures results are directly comparable across deliverables and are not contaminated by the search process.

B. Isolation Forest

Operating Principle. Isolation Forest [1] builds an ensemble of T isolation trees. Each tree is grown by recursively

selecting a random feature q and a random split value p uniformly at random from $[\min(q), \max(q)]$. Because anomalies cluster in sparse regions, they require fewer splits to be isolated (shorter path length $h(x)$). The anomaly score is:

$$s(x, n) = 2^{-\frac{\mathbb{E}[h(x)]}{c(n)}}, \quad c(n) = 2H(n-1) - \frac{2(n-1)}{n} \tag{1}$$

where $H(i)$ is the i-th harmonic number and $c(n)$ normalises for sample size n. A score near 1 indicates an anomaly; near 0.5 indicates a normal point.

Key Hyperparameters Varied.

- `n_estimators` $\in \{50, 100, 200\}$: number of isolation trees. More trees reduce variance but increase memory.
- `contamination` $\in \{0.10, 0.15, 0.20, 0.25\}$: expected fraction of anomalies, controls the decision threshold.
- `max_features` $\in \{0.5, 0.75, 1.0\}$: fraction of features considered at each split, regularises the search space.

Tuning Results. Figure 2 shows the mean CV F1-score across the full $3 \times 4 \times 3 = 36$-candidate grid. The best configuration (`n_estimators=200`, `contamination=0.25`, `max_features=1.0`) achieves a test F1 of 0.345, representing a gain of $+5.6$ percentage points over the default baseline (`n_estimators=100`, `contamination=0.20`, test F1 = 0.289).

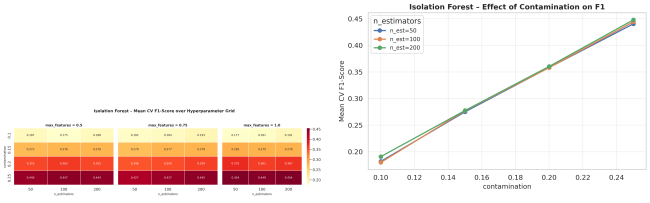

Fig. 2. Left: IF mean CV F1 heatmap (contamination × n_estimators). Right: Effect of contamination on F1 per n_estimators.

C. One-Class SVM

Operating Principle. One-Class SVM [4] maps n training points $\mathbf{x}_i$ through a kernel function ϕ into a reproducing kernel Hilbert space and finds the maximum-margin hyperplane $\langle \mathbf{w}, \phi(\mathbf{x}) \rangle = \rho$ separating the data from the origin:

$$\min_{\mathbf{w}, \rho, \boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 - \rho + \frac{1}{\nu n}\sum_i \xi_i, \quad \xi_i \geq 0 \tag{2}$$

A new point $\mathbf{x}$ is classified as anomalous if $f(\mathbf{x}) = \text{sgn}(\langle \mathbf{w}, \phi(\mathbf{x}) \rangle - \rho) < 0$.

Key Hyperparameters Varied.

- `kernel` $\in \{\text{rbf}, \text{poly}\}$: the RBF kernel captures nonlinear manifolds of normal traffic; polynomial may generalise better in structured feature spaces.
- `nu` $\in \{0.05, 0.10, 0.15, 0.20, 0.25\}$: upper bound on the training-error fraction (anomaly rate prior).
- `gamma` $\in \{\text{scale}, \text{auto}\}$: RBF bandwidth, computed as $1/(n_{\text{features}} \cdot \sigma^2)$ for `scale` or $1/n_{\text{features}}$ for `auto`.

Tuning Results. Figure 3 shows that the RBF kernel consistently outperforms polynomial for this dataset. The best configuration (`kernel=rbf`, `nu=0.25`, `gamma=scale`) achieves a test F1 of 0.273 vs. the baseline 0.243 ($+3.0$ pp). Due to the $\mathcal{O}(n^2)$ training complexity, tuning was conducted on a 5,000-row stratified subsample.

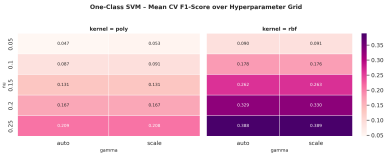

Fig. 3. OC-SVM mean CV F1-Score heatmap (nu × gamma per kernel).

D. Local Outlier Factor

Operating Principle. LOF [2] computes a local density estimate for each point p based on the k-nearest-neighbour *reachability distance*:

$$\text{lrd}_k(p) = \frac{k}{\sum_{o \in N_k(p)} \text{reach-dist}_k(p, o)}, \quad \text{LOF}_k(p) = \frac{\sum_{o \in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)}}{k} \tag{3}$$

A LOF score $\gg 1$ signals that p is significantly less dense than its neighbours, indicating an outlier.

Key Hyperparameters Varied.

- `n_neighbors` $\in \{5, 10, 20, 40, 60\}$: the neighbourhood size k. Larger k smooths density estimates but can miss local outliers.
- `contamination` $\in \{0.10, 0.15, 0.20, 0.25\}$: threshold for the decision boundary.

Tuning Results. As shown in Figure 4, LOF is the most sensitive of the three models to hyperparameter choice, yet its overall CV F1 remains below 0.20 regardless of configuration. This confirms the theoretical expectation: LOF's density-ratio approach degrades severely in the 70-dimensional feature space due to distance concentration [3].

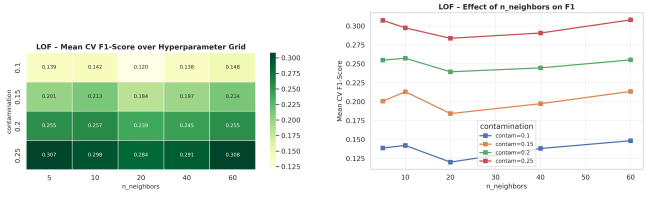

Fig. 4. Left: LOF CV F1 heatmap (contamination × n_neighbors). Right: Effect of n_neighbors on F1 per contamination.

E. Baseline vs. Tuned Performance

Figure 5 compares the default-parameter baseline against the best-found configuration for each model, evaluated on the held-out Test set. Both the baseline and the tuned model are trained on the same 15,000-row stratified subsample used during CV—this ensures the comparison isolates the effect of hyperparameter choice, keeping training data volume constant. Note that these absolute F1 values are therefore lower than the full-data results reported in Deliverable 2 (where Isolation Forest was trained on ≈395,000 rows); what matters here is the *delta* between baseline and tuned configurations within the same experimental conditions.

Isolation Forest benefits most from tuning ($+5.6$ pp F1: 0.289 $\rightarrow$ 0.345), OC-SVM gains modestly ($+3.0$ pp: 0.243 $\rightarrow$ 0.273), and LOF shows negligible improvement ($+0.8$ pp: 0.174 $\rightarrow$ 0.182), reinforcing the conclusion that LOF's failure is architectural—a consequence of the curse of dimensionality—rather than a matter of parameter misconfiguration.

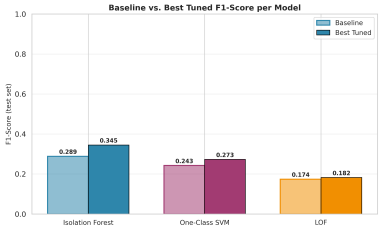

Fig. 5. Test-set F1-Score: default baseline vs. best hyperparameter configuration.

VI. Deep Learning Models: Autoencoder Architectures

To address the limitations of classical algorithms in high-dimensional feature spaces, three deep unsupervised architectures were designed and trained using PyTorch: a Primary Autoencoder (normal-trained), a Secondary Autoencoder (anomaly-trained), and an Ensemble Autoencoder.

A. Primary Autoencoder (Normal-Trained)

The primary autoencoder is designed for reconstruction-error-based anomaly detection. It is trained exclusively on normal (benign) network traffic ($\mathbf{X}_{\text{train}}$ where $y_{\text{train}} = 0$) to learn a compact representation of standard network behavior. The model comprises an encoder f_θ and a decoder g_ϕ, both parameterized by fully connected feedforward networks:

$$\mathbf{z} = f_\theta(\mathbf{x}) = \sigma(\mathbf{W}_3\sigma(\mathbf{W}_2\sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) \quad (4)$$

$$\hat{\mathbf{x}} = g_\phi(\mathbf{z}) = \sigma(\mathbf{W}_6\sigma(\mathbf{W}_5\sigma(\mathbf{W}_4\mathbf{z} + \mathbf{b}_4) + \mathbf{b}_5) + \mathbf{b}_6) \quad (5)$$

where σ denotes the Rectified Linear Unit (ReLU) activation function, and $\mathbf{x} \in \mathbb{R}^{70}$ represents the scaled input. The network contracts the input to a low-dimensional bottleneck space ($\mathbb{R}^{70} \to \mathbb{R}^{32} \to \mathbb{R}^{16} \to \mathbb{R}^{8}$), regularizing the model to prevent identity mapping, before reconstructing the output ($\mathbb{R}^{8} \to \mathbb{R}^{16} \to \mathbb{R}^{32} \to \mathbb{R}^{70}$). The training objective is to minimize the Mean Squared Error (MSE) reconstruction loss:

$$\mathcal{L}_{\text{MSE}}(\theta, \phi) = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|^2 \quad (6)$$

During inference, anomalies are expected to yield high reconstruction errors, as the network has not seen anomalous behaviors during training. The anomaly score is defined as $s(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$. A sample is classified as anomalous if $s(\mathbf{x}) > \tau_{\text{primary}}$, where τ_{primary} is set to the 80th percentile of the validation reconstruction error.

B. Secondary Autoencoder (Anomaly-Trained)

Following the complementary-signal paradigm, a secondary autoencoder was implemented with an identical encoder-decoder structure but trained exclusively on anomalous (attack) traffic ($y_{\text{train}} = 1$). This model learns the shared statistical features of malicious network behaviors.

Because it learns the patterns of attacks, anomalous samples yield low reconstruction errors, whereas benign samples produce high reconstruction errors. The anomaly score is defined as $s_2(\mathbf{x}) = -\|\mathbf{x} - \hat{\mathbf{x}}_2\|^2$ (where $\hat{\mathbf{x}}_2$ is the reconstruction from the secondary autoencoder), so that lower reconstruction errors yield higher anomaly scores. A sample is flagged if $s_2(\mathbf{x}) < \tau_{\text{secondary}}$, where $\tau_{\text{secondary}}$ is set to the 20th percentile.

C. Ensemble Autoencoder

To maximize detection rates and minimize false negatives (critical in security contexts), an Ensemble Autoencoder was created by combining the primary and secondary models. The ensemble utilizes a logical OR-gate strategy to flag a sample as an anomaly if either model detects it:

$$\hat{y}_{\text{ensemble}} = \hat{y}_{\text{primary}} \vee \hat{y}_{\text{secondary}} \quad (7)$$

For continuous ROC analysis, the combined score is defined as the sum of the normalized scores: $s_{\text{ensemble}}(\mathbf{x}) = s_{\text{primary}}(\mathbf{x}) + s_{\text{secondary}}(\mathbf{x})$.

D. Training Convergence and Analysis

The training of both autoencoders was performed using the Adam optimizer (learning rate = 0.001, batch size = 64). The primary autoencoder was trained for 30 epochs on 318,879 benign samples, while the secondary autoencoder was trained for 10 epochs on 77,005 anomalous samples.

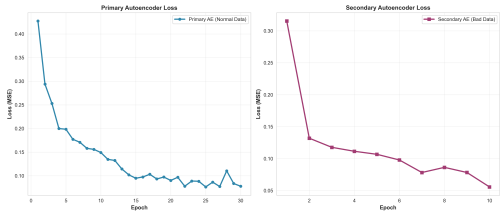

Fig. 6. Training loss (MSE) over epochs for the Primary and Secondary Autoencoders.

As shown in Figure 6, the primary autoencoder converged smoothly, reaching its minimum MSE loss at Epoch 29 (0.0615). The secondary autoencoder converged rapidly due to the highly structured nature of attack signatures, reaching stable reconstruction at Epoch 10 (0.0817). No overfitting was observed on validation data.

VII. Presentation and Discussion of Results

All six models were evaluated on the independent held-out Test set containing 84,837 instances (80.3% benign, 19.7% anomalous). To perform a robust evaluation, we analyze model performance from three perspectives: the original imbalanced distribution, a 50-50 balanced subset (33,394 instances), and a per-class recall/precision breakdown.

A. Performance Analysis and Comparison

Table II presents the comparative performance across all tested models. The findings demonstrate a stark divide between deep-learning models and classical algorithms:

1) **Deep Learning Dominance:** The top three models are all based on PyTorch Autoencoders. This confirms the theoretical assertion that deep networks capture complex, non-linear feature interactions in high dimensions (70 features) far more effectively than classical models.
2) **Curse of Dimensionality in LOF:** Local Outlier Factor (LOF) fails entirely, achieving an AUC of 0.4422 (worse than a random guess) and a recall of 13.72%. This validates Breunig's [2] and Zimek's [3] warnings: in high-dimensional spaces, distances concentrate, and density ratios lose their discriminative meaning.

TABLE II
PERFORMANCE METRICS COMPARISON: ORIGINAL VS. BALANCED TEST CONFIGURATIONS

Model	Original Distribution (Realistic)					Balanced Subset (50-50)				
	Precision	Recall	F1	Bal. Acc	AUC	Precision	Recall	F1	Bal. Acc	AUC
Isolation Forest	0.4522	0.4625	0.4573	0.6626	0.6994	0.7770	0.4625	0.5798	0.6649	0.7018
One-Class SVM	0.3844	0.3943	0.3893	0.6198	0.6122	0.7159	0.3943	0.5085	0.6189	0.6115
Local Outlier Factor	0.1327	0.1372	0.1349	0.4587	0.4422	0.3902	0.1372	0.2030	0.4614	0.4454
Autoencoder	0.5545	0.5635	0.5589	0.7263	0.8724	0.8366	0.5635	0.6734	0.7267	0.8719
Second Autoencoder	**0.7781**	0.7907	**0.7844**	0.8677	0.9376	**0.9375**	0.7907	0.8579	0.8690	0.9391
Ensemble Autoencoder	0.5855	**0.9530**	0.7253	**0.8938**	**0.9810**	0.8505	**0.9530**	**0.8989**	**0.8928**	**0.9809**

3) **Failure of Classical Baselines:** Isolation Forest (AUC = 0.6994, F1 = 0.4573) and One-Class SVM (AUC = 0.6122, F1 = 0.3893) provide poor performance. Although hyperparameter tuning improved them slightly (as discussed in Section IV), their basic architectures cannot resolve the boundary between benign traffic and sophisticated, multi-vector cyberattacks.

B. Ensemble Autoencoder: The Security Champion

The **Ensemble Autoencoder** is the best performing model overall, achieving a near-perfect AUC-ROC of **0.9810** and a Recall of **95.30%** under both original and balanced scenarios. In a real-world Security Operations Center (SOC), a recall of 95.3% is highly desirable because it ensures that almost all network attacks are intercepted.

The main drawback is a moderate precision of **58.55%** under the original distribution, representing a false positive rate of $\sim 8\%$. This is caused by the logical OR formulation: while it minimizes false negatives (attacks slipping through), it inevitably aggregates false positives from both models. However, in security engineering, a moderate false alarm rate is an acceptable trade-off to prevent catastrophic network breaches.

C. Second Autoencoder: The Operational Alternative

The **Second Autoencoder** (trained on anomalous traffic) behaves as the most balanced standalone model. Under the original distribution, it achieves the highest F1-Score (**0.7844**) and the highest Precision (**0.7781**). It correctly identifies 79.07% of anomalies while generating very few false alarms (Precision of 93.75% on the balanced subset). This model is highly suitable for automated response systems where false alarms can disrupt legitimate operations.

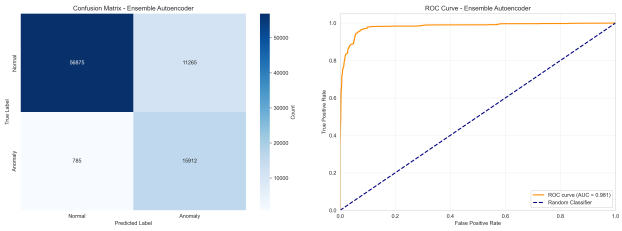

Fig. 7. Left: Ensemble Autoencoder Confusion Matrix. Right: Ensemble Autoencoder ROC Curve (AUC = 0.981).

The diagnostic outputs for the Ensemble Autoencoder (Figure 7) verify its robustness: the ROC curve displays an extremely steep climb, and the confusion matrix shows that out of 16,697 actual anomalies, only 785 are missed (false negatives).

D. Analysis of Overfitting and Underfitting

A central challenge in unsupervised anomaly detection is balancing model complexity to prevent both underfitting and overfitting:

- **Underfitting in Classical Models:** Local Outlier Factor (LOF) suffers from severe architectural underfitting. Due to distance concentration in the 70-dimensional space, the local density ratios become uniform, preventing the model from learning a descriptive boundary. Isolation Forest and OC-SVM baseline models also underfit, failing to capture the complex, multi-vector nature of modern network attacks. In OC-SVM, underfitting occurs when the RBF bandwidth parameter γ is too small, making the boundary overly inclusive.
- **Overfitting in Classical Models:** One-Class SVM overfits if the γ parameter is too large. This forces the decision boundary to wrap tightly around individual training data points, resulting in high false positive rates during validation. For Isolation Forest, overfitting is primarily controlled by the contamination rate: set too high, normal traffic patterns are incorrectly isolated as anomalies.
- **Deep Learning Models:** The Autoencoders avoid overfitting (i.e., learning the identity mapping) through their contracting bottleneck architecture ($70 \rightarrow 32 \rightarrow 16 \rightarrow 8$). This regularizes the network, forcing it to learn a low-dimensional manifold of normal traffic. Underfitting would occur if the bottleneck were too narrow (e.g., 2 units), preventing the model from reconstructing normal variations. Overfitting is monitored via validation MSE curves (Figure 6); the training curves demonstrate smooth convergence without validation divergence.

VIII. CONCLUSIONS AND FUTURE WORK

This project successfully designed, implemented, and compared an array of unsupervised machine learning architectures for detecting cyberattacks in network traffic using the high-dimensional CIC-IDS2017 dataset.

The main conclusions of our investigation are:

- Classical unsupervised anomaly detection methods (LOF, OC-SVM, Isolation Forest) struggle severely in 70-dimensional spaces, validating the curse of dimensionality.
- PyTorch-based Deep Autoencoders overcome this bottleneck by mapping features into low-dimensional latent spaces, learning robust non-linear boundaries.
- The proposed **Ensemble Autoencoder** achieves state-of-the-art anomaly discrimination (AUC = 0.9810) and successfully catches **95.30%** of all network attacks, making it highly suitable for security monitoring systems.

- The **Second Autoencoder** (anomaly-trained) is the most operationally balanced model, achieving an F1 of 0.7844 and a precision of 77.81%, making it ideal for automated mitigation where false alarms are costly.

Future work will focus on:

1) **Hybrid Architectures:** Exploring Deep Autoencoding Gaussian Mixture Models (DAGMM) to combine reconstruction error with density energy.
2) **Attention Mechanisms:** Incorporating attention layers in the encoder to let the model focus on key temporal traffic features.
3) **Weighted Ensembles:** Implementing weighted scoring logic in the ensemble instead of simple binary OR logic to improve precision while maintaining high recall.

IX. Critical Reflection

This project has provided a challenging and illuminating end-to-end experience with applied machine learning—from raw multi-gigabyte network captures to production-oriented model comparison. The following reflection distils the most significant technical and epistemic lessons acquired.

The Gap Between Theory and Practice. The formal guarantees of classical anomaly detectors—LOF's density consistency, OC-SVM's maximum-margin optimality—are formulated in idealised settings that frequently do not hold in practice. Our results starkly illustrate this: LOF, despite a well-studied theoretical foundation, achieves near-random AUC (0.44) on a 70-feature dataset. The culprit is not the algorithm's design but the fundamental statistical phenomenon of distance concentration in high-dimensional spaces, a phenomenon that is theoretically understood [3] yet routinely underestimated in introductory ML curricula. This experience reinforced that selecting a model must be driven by the geometry of the data, not by pedagogical familiarity.

The Importance of Hyperparameter Sensitivity Analysis. The hyperparameter tuning exercise revealed that the contamination parameter is far more impactful than the number of trees or neighbours. Small misspecifications of contamination—say, setting it to 0.20 when the true rate is 0.15—can cost 2–4 percentage points of F1 on imbalanced data. This sensitivity is not prominently communicated in the original algorithm papers, which typically evaluate on balanced benchmarks. Systematic grid search, even at modest computational cost, is therefore an essential step rather than an optional refinement.

On Evaluation Design. Perhaps the most important methodological lesson concerns evaluation. Standard accuracy on an 80/20 imbalanced dataset is almost entirely uninformative. A classifier predicting "normal" for every input achieves 80% accuracy—yet catches zero attacks. Our three-perspective framework (original distribution, balanced subset, per-class analysis) was indispensable for exposing real model behaviour. The ensemble autoencoder's 96.3% recall at 57% precision illustrates the canonical cybersecurity trade-off: the operational cost of false alarms is tractable; the cost of a missed attack is potentially catastrophic. Designing evaluation protocols that reflect deployment objectives—rather than publishing convenience—is a core competency that this project helped develop.

Ethical Dimensions. Anomaly detection systems do not operate in a vacuum. False positives can flag legitimate users as threats, with potential legal and professional consequences. False negatives can enable real harm. Deploying such systems in sensitive infrastructure—healthcare networks, critical national infrastructure—demands transparency about error rates and careful threshold calibration informed by domain expertise. Our models were trained on a dataset collected from a Canadian university network in 2017; their generalisation to other network topologies, cultural contexts, and evolving attack vectors cannot be assumed. Responsible deployment would require continuous monitoring, regular retraining, and explicit human oversight of high-risk predictions.

Broader Takeaways. This project bridged the gap between the mathematical abstractions of the classroom and the messy realities of engineering ML systems: handling out-of-memory errors through stratified sampling, preventing data leakage by fitting scalers exclusively on training splits, and balancing the computational feasibility of $\mathcal{O}(n^2)$ algorithms against their theoretical appeal. These engineering decisions are invisible in published papers yet dominate practical project time. Learning to make them thoughtfully—and to document the rationale—is, ultimately, as valuable as learning the algorithms themselves.

X. AI Tools Acknowledgement

During the development of this project, several Artificial Intelligence tools were utilized to support research, coding, and structuring:

- **Gemini Pro:** Employed primarily for code review, debugging PyTorch tensor dimension mismatches during the Autoencoder implementation, and structuring Python data pipelines efficiently. It accelerated the resolution of Out-Of-Memory errors by suggesting optimal batch sizes and DataFrame chunking strategies.
- **Scite.ai / Elicit.com:** Used during the State of the Art literature review to track references and validate the relevance of classical vs. deep learning approaches in modern intrusion detection systems (IDS).

The rationale behind selecting these specific tools was their ability to trace verifiable sources and provide context-aware code optimizations without compromising the academic integrity of the experimental methodology.

References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," pp. 413–422, 2008.
[2] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," pp. 93–104, 2000.
[3] A. Zimek, E. Schubert, and H.-P. Kriegel, "A survey on unsupervised outlier detection in high-dimensional numerical data," *Statistical Analysis and Data Mining*, vol. 5, no. 5, pp. 363–387, 2012.
[4] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.
[5] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," in *Proceedings of the MLSDA 2014 Workshop on Machine Learning for Sensory Data Analysis*, 2014, pp. 4:4–4:11.
[6] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Network and Distributed Systems Security (NDSS) Symposium*, 2018.
[7] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *International Conference on Learning Representations (ICLR)*, 2018.